

MiniPython – A Small Subset of Python Language

1 INTRODUCTION

MiniPython is a toy language for the [COSE212](#) course at Korea University. MiniPython is inspired by the Python programming language.¹ The name MiniPython indicates that it is a minimal subset of the Python language, and it supports the following features:

- **pass statements** (pass)
- **variable assignment statement** ($x = e$)
- **set item** ($x[e] = e$)
- **if-statements** (if-elif-else)
- **while-statements** (while)
- **for-statements** (for-in)
- **loop control statements** (break and continue)
- **try and raise statements** (try-except and raise)
- **function definitions** (def $f(\dots):$)
- **return statements** (return)
- **yield and yield from statements** (yield and yield from)
- **none value** (None)
- **number (integer) values** (0, 1, -1, 2, -2, 3, -3, ...)
- **boolean values** (True and False)
- **arithmetic operators**: negation (-), addition (+), subtraction (-), multiplication (*), division (/), and modulo (%)
- **logical operators**: conjunction (and), disjunction (or), and negation (not)
- **comparison operators**: equality (==, !=, is, and is not) and relational (<, >, <=, and >=)
- **lists** ([$e, \dots, e$])
- **list append method** ($e.append(e)$)
- **get item** ($x[e]$)
- **lambda functions** (lambda $\dots: e$)
- **function applications** ($e(\dots)$)
- **conditional expressions** ($e \text{ if } e \text{ else } e$)
- **iter function** (iter(e))
- **next function** (next(e))

This document is the specification of MiniPython. First, Section 2 describes the concrete syntax, and Section 3 describes the abstract syntax with the desugaring rules. Then, Section 4 describes the small-step operational semantics of MiniPython.

2 CONCRETE SYNTAX

The concrete syntax of MiniPython is written in a variant of the extended Backus–Naur form (EBNF). The notation $\langle nt \rangle$ denotes a nonterminal, and "t" denotes a terminal. We use ? to denote an optional element and + (or *) to denote one or more (or zero or more) repetitions of the preceding element. We use **but not** to denote a set difference to exclude some strings from a producible set of strings. We omit some obvious terminals using the ellipsis (...) notation. The $\langle \text{INDENT} \rangle$ and $\langle \text{DEDENT} \rangle$ denote the increase and decrease of the indentation level, respectively.

¹<https://www.python.org/>

```

// basic elements
<digit>      ::= "0" | "1" | "2" | ... | "9"
<number>     ::= "-"? <digit>+
<alphabet>   ::= "A" | "B" | "C" | ... | "Z" | "a" | "b" | "c" | ... | "z"
<idstart>    ::= <alphabet> | "_"
<idcont>     ::= <alphabet> | "_" | <digit>
<keyword>    ::= "False" | "None" | "True" | "and" | "break" | "continue" | "def"
               | "elif" | "else" | "except" | "for" | "from" | "if" | "in"
               | "lambda" | "not" | "or" | "pass" | "raise" | "return" | "try"
               | "while" | "yield"
<id>         ::= <idstart> <idcont>* butnot <keyword>
<program>    ::= <stmt>* <expr> // programs
<block>      ::= <INDENT> <stmt>+ <DEDENT> | <stmt> // blocks
// statements
<stmt> ::= "pass" | <expr> | <id> "=" <expr> | <expr> "[" <expr> "]" "=" <expr>
           // if-statements
           | "if" <expr> ":" <block> [ "elif" <expr> ":" <block> ]*
           [ "else" ":" <block> ]
           // while-statements and for-statements
           | "while" <expr> ":" <block> | "for" <id> "in" <expr> ":" <block>
           // loop control statements, try statements, and raise statements
           | "break" | "continue" | "try" ":" <block> "except" ":" <block>
           // function definitions
           | "def" <id> "(" [ <id> [ ",", <id> ]* ] ")" ":" <block>
           // raise, return, yield, and yield from statements
           | "raise" | "return" <expr> | "yield" <expr> | "yield" "from" <expr>
// expressions
<expr> ::= "None" | <number> | "True" | "False" | <id>
           // unary/binary operators and parentheses
           | <uop> <expr> | <expr> <bop> <expr> | "(" <expr> ")"
           // lists
           | "[" [ <expr> [ ",", <expr> ]* ] "]"
           // list append method and get item
           | <expr> "." "append" "(" <expr> ")" | <expr> "[" <expr> "]"
           // lambda functions
           | "lambda" [ <id> [ ",", <id> ]* ] ":" <expr>
           // function applications
           | <expr> "(" [ <expr> [ ",", <expr> ]* ] ")"
           // conditional expressions
           | <expr> "if" <expr> "else" <expr>
           // iter function and next function
           | "iter" "(" <expr> ")" | "next" "(" <expr> ")"
// operators
<uop> ::= "-" | "not"
<bop> ::= "+" | "-" | "*" | "/" | "%" | "and" | "or"
           | "==" | "!=" | "is" | "is not" | "<" | "<=" | ">" | ">="

```

The precedence and associativity of operators are defined as follows:

Description	Operator	Precedence	Associativity
Method Call	<code>_.append</code>	10	left
Prefix Unary	<code>-</code>	9	right
Multiplicative	<code>*, //, %</code>	8	left
Additive	<code>+, -</code>	7	
Comparison	<code>==, !=, is, is not, <, <=, >, >=</code>	6	
Logical Negation	<code>not</code>	5	
Logical Conjunction	<code>and</code>	4	
Logical Disjunction	<code>or</code>	3	-
Conditional Expression	<code>if-else</code>	2	
Lambda Function	<code>lambda</code>	1	

3 ABSTRACT SYNTAX

The abstract syntax of MiniPython is defined as follows:

Programs	$p ::= S; \dots; S; e$ (Program)	Blocks	$B ::= S \mid \{S; \dots; S\}$ (Block)
Statements	$S ::= \text{pass}$ (SPass) <code>continue</code> (SContinue) $\mid e$ (SEExpr) <code>try B except B</code> (STry) $\mid x = e$ (SAssign) <code>raise</code> (SRaise) $\mid e[e] = e$ (SSetItem) <code>def x (x, \dots, x) B</code> (SDef) $\mid \text{if } e \text{ B else } B$ (SIf) <code>return e</code> (SReturn) $\mid \text{while } e \text{ B}$ (SWhile) <code>yield e</code> (SYield) $\mid \text{break}$ (SBreak)		
Expressions	$e ::= \text{None}$ (ENone) $e[e]$ (EGetItem) $\mid n$ (ENum) $\lambda(x, \dots, x).e$ (ELambda) $\mid b$ (EBool) $e(e, \dots, e)$ (EApp) $\mid x$ (EId) $e \text{ if } e \text{ else } e$ (ECond) $\mid e \oplus e$ (EOp) <code>iter(e)</code> (EIter) $\mid [e, \dots, e]$ (EList) <code>next(e)</code> (ENext) $\mid e.append(e)$ (EAppend)		
Operators	$\oplus ::= +$ (Add) $*$ (Mul) $//$ (Div) $\%$ (Mod) $<$ (Lt) $<=$ (Lte) $==$ (Eq) <code>is</code> (Is)		
where	$\left\{ \begin{array}{l} \text{Booleans } \mathbb{B} \ni b ::= \text{True} \mid \text{False} \text{ (Boolean)} \\ \text{Identifiers } x \in \mathbb{X} \text{ (String)} \\ \text{Numbers } n, m, k \in \mathbb{Z} \text{ (BigInt)} \end{array} \right.$		

The semantics of the remaining cases are defined with the following desugaring rules:

$$\begin{aligned}
\mathcal{D}[-e] &= \mathcal{D}[e] * (-1) & \mathcal{D}[e_1 \text{ is not } e_2] &= \mathcal{D}[\text{not } (e_1 \text{ is } e_2)] \\
\mathcal{D}[e_1 - e_2] &= \mathcal{D}[e_1] + \mathcal{D}[-e_2] & \mathcal{D}[e_1 \text{ and } e_2] &= \text{if } (\mathcal{D}[e_1]) \mathcal{D}[e_2] \text{ else False} \\
\mathcal{D}[e_1 > e_2] &= \mathcal{D}[\text{not } (e_1 \leq e_2)] & \mathcal{D}[e_1 \text{ or } e_2] &= \text{if } (\mathcal{D}[e_1]) \text{ True else } \mathcal{D}[e_2] \\
\mathcal{D}[e_1 \geq e_2] &= \mathcal{D}[\text{not } (e_1 < e_2)] & \mathcal{D}[\text{not } e] &= \text{if } (\mathcal{D}[e]) \text{ False else True} \\
\mathcal{D}[e_1 != e_2] &= \mathcal{D}[\text{not } (e_1 == e_2)] & \mathcal{D}[\text{yield from } e] &= \mathcal{D}[\text{for } x \text{ in } e \text{ yield } x]
\end{aligned}$$

$$\begin{aligned}
\mathcal{D}[\text{if } e \text{ B elif } e_1 B_1 \dots \text{elif } e_n B_n \text{ else } B'] &= \mathcal{D}[\text{if } e \text{ B else if } e_1 B_1 \dots \text{else if } e_n B_n \text{ else } B'] \\
\mathcal{D}[\text{if } e \text{ B elif } e_1 B_1 \dots \text{elif } e_n B_n] &= \mathcal{D}[\text{if } e \text{ B else if } e_1 B_1 \dots \text{else if } e_n B_n \text{ else pass}]
\end{aligned}$$

$$\mathcal{D}[\text{for } x \text{ in } e \text{ B}] = \mathcal{D}[\underline{x} = e; \text{ while True } \{ \text{try } x = \text{next}(\underline{x}) \text{ except break; } B \}]$$

where $\underline{x}$ denotes a fresh temporary variable. All the omitted cases recursively apply the desugaring rule to their sub-expressions. For example, $\mathcal{D}[e_1 + e_2] = \mathcal{D}[e_1] + \mathcal{D}[e_2]$.

4 SEMANTICS

We use the following notations in the semantics:

States $\langle \kappa \parallel s \parallel H \parallel M \rangle \in \mathbb{K} \times \mathbb{S} \times \mathbb{H} \times \mathbb{M}$ (State) Continuations $\mathbb{K} \ni \kappa ::= \square \mid i :: \kappa$ (Cont)

Instructions	$i ::= (\sigma \vdash_B B)$	(IBlock)	$\mid \text{jmp-if}[\psi]$	(IJmpIf)
	$\mid (\sigma \vdash_S S)$	(ISmt)	$\mid \text{jmp}[c]$	(IJmp)
	$\mid (\sigma \vdash_e e)$	(IExpr)	$\mid \text{raise}(\omega)$	(IRaise)
	$\mid (\oplus)$	(IBop)	$\mid \text{call}[n]$	(ICall)
	$\mid \text{write}[a]$	(IWrite)	$\mid \text{return}$	(IReturn)
	$\mid \text{get-item}$	(IGetItem)	$\mid \text{yield}$	(IYield)
	$\mid \text{set-item}$	(ISetItem)	$\mid \text{iter}$	(IIter)
	$\mid \text{list}[n]$	(IList)	$\mid \text{next}$	(INext)
	$\mid \text{append}$	(IAppend)	$\mid \text{drop}$	(IDrop)

Value Stacks $s ::= \blacksquare \mid v :: s$ (Stack)

Control Handlers	$H \in \mathbb{H} = \mathbb{C} \xrightarrow{\text{fin}} \Psi$ (Handler)	Environments	$\sigma \in \mathbb{X} \xrightarrow{\text{fin}} \mathbb{A}$ (Env)
Memories	$M \in \mathbb{M} = \mathbb{A} \xrightarrow{\text{fin}} \mathbb{V}$ (Mem)	Addresses	$a \in \mathbb{A}$ (Addr)

Errors $\omega ::= \text{RuntimeError} \mid \text{ZeroDivisionError} \mid \text{TypeError} \mid \text{IndexError}$
 $\mid \text{StopIteration} \mid \text{NameError}[x]$

Values	$v ::= \text{None}$	(NoneV)	$\mid \langle \lambda(x, \dots, x).B, \sigma \rangle$	(CloV)
	$\mid n$	(NumV)	$\mid \langle \lambda*(x, \dots, x).B, \sigma \rangle$	(GenV)
	$\mid b$	(BoolV)	$\mid \langle \kappa \parallel s \parallel H \rangle$	(ContV)
	$\mid a$	(AddrV)	$\mid \text{iter}[a, n]$	(IterV)
	$\mid [v, \dots, v]$	(ListV)		

Continuation Values $\psi, \langle \kappa \parallel s \parallel H \rangle \in \Psi = \mathbb{K} \times \mathbb{S} \times \mathbb{H}$ (KValue)

Controls	$c ::= \text{return}$	(Return)	$\mid \text{throw}$	(Throw)
	$\mid \text{break}$	(Break)	$\mid \text{finally}$	(Finally)
	$\mid \text{continue}$	(Continue)	$\mid \text{yield}$	(Yield)

The small-step operational semantics of MiniPython is defined in the following form of the small-step relation ($\rightarrow$):

$$\boxed{\langle \kappa \parallel s \parallel H \parallel M \rangle \rightarrow \langle \kappa \parallel s \parallel H \parallel M \rangle}$$

The evaluation result of a program $p = S_1; \dots; S_n; e$ is the value v if

$$\langle (\sigma \vdash S_1) :: \dots :: (\sigma \vdash S_n) :: (\sigma \vdash e) :: \square \parallel \blacksquare \parallel \emptyset \parallel \emptyset \rangle \rightarrow^* \langle \square \parallel v :: \blacksquare \parallel _ \parallel _ \rangle$$

$$\text{where } \begin{cases} \{x_1, \dots, x_k\} &= \text{locals}(\{S_1, \dots, S_n\}) \\ a_1, \dots, a_k &= (\text{distinct fresh addresses}) \\ \sigma &= [x_1 \mapsto a_1, \dots, x_k \mapsto a_k] \\ M &= [a_1 \mapsto \text{None}, \dots, a_k \mapsto \text{None}] \end{cases}$$

4.1 Small-Step Relations for **ISmt**

S _{Pass}	$\langle (\sigma \vdash_S \text{pass}) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle \kappa \parallel s \parallel H \parallel M \rangle$
S _{Expr}	$\langle (\sigma \vdash_S e) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle (\sigma \vdash_e e) :: \text{drop} :: \kappa \parallel s \parallel H \parallel M \rangle$
S _{Assign}	$\langle (\sigma \vdash_S x = e) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle (\sigma \vdash_e e) :: \text{write}[\sigma(x)] :: \kappa \parallel s \parallel H \parallel M \rangle$
S _{SetItem}	$\langle (\sigma \vdash_S e_0[e_1] = e_2) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle (\sigma \vdash_e e_2) :: (\sigma \vdash_e e_0) :: (\sigma \vdash_e e_1) :: \text{set-item} :: \kappa \parallel s \parallel H \parallel M \rangle$
S _{If}	$\langle (\sigma \vdash_S \text{if } e \text{ } B_0 \text{ else } B_1) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle (\sigma \vdash_e e) :: \text{jmp-if}[\langle (\sigma \vdash_B B_0) :: \kappa \parallel s \parallel H \rangle] :: (\sigma \vdash_B B_1) :: \kappa \parallel s \parallel H \parallel M \rangle$
S _{While}	$\langle (\sigma \vdash_S \text{while } e \text{ } B) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle (\sigma \vdash_e e) :: \text{jmp-if}[\psi_{\text{body}}] :: \kappa \parallel s \parallel H \parallel M \rangle$
where	$\begin{cases} \psi_{\text{body}} &= \langle (\sigma \vdash_B B) :: (\sigma \vdash_S \text{while } e \text{ } B) :: \kappa \parallel s \parallel H_{\text{body}} \rangle \\ H_{\text{body}} &= H[\text{continue} \mapsto \psi_{\text{continue}}, \text{break} \mapsto \psi_{\text{break}}] \\ \psi_{\text{continue}} &= \langle (\sigma \vdash_S \text{while } e \text{ } B) :: \kappa \parallel s \parallel H \rangle \\ \psi_{\text{break}} &= \langle \kappa \parallel s \parallel H \rangle \end{cases}$
S _{Break}	$\langle (\sigma \vdash_S \text{break}) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle \text{jmp}[\text{break}] :: \kappa \parallel s \parallel H \parallel M \rangle$
S _{Continue}	$\langle (\sigma \vdash_S \text{continue}) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle \text{jmp}[\text{continue}] :: \kappa \parallel s \parallel H \parallel M \rangle$
S _{Try}	$\langle (\sigma \vdash_S \text{try } B_0 \text{ except } B_1) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle (\sigma \vdash_B B_0) :: \text{jmp}[\text{finally}] :: \square \parallel s \parallel H_{\text{body}} \parallel M \rangle$
where	$\begin{cases} H_{\text{body}} &= H[\text{raise} \mapsto \psi_{\text{raise}}, \text{finally} \mapsto \psi_{\text{finally}}] \\ \psi_{\text{raise}} &= \langle (\sigma \vdash_B B_1) :: \kappa \parallel s \parallel H \rangle \\ \psi_{\text{finally}} &= \langle \kappa \parallel s \parallel H \rangle \end{cases}$
S _{Raise}	$\langle (\sigma \vdash_S \text{raise}) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle \text{raise}(\text{RuntimeError}) :: \square \parallel s \parallel H \parallel M \rangle$
S _{Def}	$\langle (\sigma \vdash_S \text{def } x_0 \text{ } (x_1, \dots, x_n) \text{ } B) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle \text{write}[\sigma(x_0)] :: \kappa \parallel a :: s \parallel H \parallel M[a \mapsto v] \rangle$
	where $a \notin \text{Domain}(M)$ and $v = \begin{cases} \langle \lambda^*(x_1, \dots, x_n).B, \sigma \rangle & \text{if } \text{hasYield}(B) \\ \langle \lambda(x_1, \dots, x_n).B, \sigma \rangle & \text{otherwise} \end{cases}$
S _{Return}	$\langle (\sigma \vdash_S \text{return } e) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle (\sigma \vdash_e e) :: \text{return} :: \kappa \parallel s \parallel H \parallel M \rangle$
S _{Yield}	$\langle (\sigma \vdash_S \text{yield } e) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle (\sigma \vdash_e e) :: \text{yield} :: \kappa \parallel s \parallel H \parallel M \rangle$

4.2 Small-Step Relations for IBlock

$$\begin{array}{l} \text{IBlock} \quad \langle (\sigma \vdash_B \{S_1; \dots; S_n\}) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle (\sigma \vdash_S S_1) :: \dots :: (\sigma \vdash_S S_n) :: \kappa \parallel s \parallel H \parallel M \rangle \end{array}$$

4.3 Small-Step Relations for IExpr

$$\begin{array}{l} \text{ENone} \quad \langle (\sigma \vdash_e \text{None}) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle \kappa \parallel \text{None} :: s \parallel H \parallel M \rangle \\ \\ \text{ENum} \quad \langle (\sigma \vdash_e n) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle \kappa \parallel n :: s \parallel H \parallel M \rangle \\ \\ \text{EBool} \quad \langle (\sigma \vdash_e b) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle \kappa \parallel b :: s \parallel H \parallel M \rangle \\ \\ \text{EId} \quad \langle (\sigma \vdash_e x) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \begin{cases} \langle \kappa \parallel M(\sigma(x)) :: s \parallel H \parallel M \rangle & \text{if } x \in \text{Domain}(\sigma) \\ \langle \text{raise}(\text{NameError}[x]) :: \square \parallel s \parallel H \parallel M \rangle & \text{otherwise} \end{cases} \\ \\ \text{EBOp} \quad \langle (\sigma \vdash_e e_0 \oplus e_1) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle (\sigma \vdash_e e_0) :: (\sigma \vdash_e e_1) :: (\oplus) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \\ \text{EList} \quad \langle (\sigma \vdash_e [e_1, \dots, e_n]) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle (\sigma \vdash_e e_1) :: \dots :: (\sigma \vdash_e e_n) :: \text{list}[n] :: \kappa \parallel s \parallel H \parallel M \rangle \\ \\ \text{EAppend} \quad \langle (\sigma \vdash_e e_0.\text{append}(e_1)) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle (\sigma \vdash_e e_0) :: (\sigma \vdash_e e_1) :: \text{append} :: \kappa \parallel s \parallel H \parallel M \rangle \\ \\ \text{EGetItem} \quad \langle (\sigma \vdash_e e_0[e_1]) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle (\sigma \vdash_e e_0) :: (\sigma \vdash_e e_1) :: \text{get-item} :: \kappa \parallel s \parallel H \parallel M \rangle \\ \\ \text{ELambda} \quad \langle (\sigma \vdash_e \lambda(x_1, \dots, x_n).e) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle \kappa \parallel a :: s \parallel H \parallel M[a \mapsto \langle \lambda(x_1, \dots, x_n).\text{return } e, \sigma \rangle] \rangle \\ \text{where } a \notin \text{Domain}(M) \\ \\ \text{EApp} \quad \langle (\sigma \vdash_e e_0(e_1, \dots, e_n)) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle (\sigma \vdash_e e_0) :: (\sigma \vdash_e e_1) :: \dots :: (\sigma \vdash_e e_n) :: \text{call}[n] :: \kappa \parallel s \parallel H \parallel M \rangle \\ \\ \text{ECond} \quad \langle (\sigma \vdash_e e_0 \text{ if } e_1 \text{ else } e_2) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle (\sigma \vdash_e e_1) :: \text{jmp-if}[\psi] :: (\sigma \vdash_e e_2) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \text{where } \psi = \langle (\sigma \vdash_e e_0) :: \kappa \parallel s \parallel H \rangle \\ \\ \text{EIter} \quad \langle (\sigma \vdash_e \text{iter}(e)) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle (\sigma \vdash_e e) :: \text{iter} :: \kappa \parallel s \parallel H \parallel M \rangle \\ \\ \text{ENext} \quad \langle (\sigma \vdash_e \text{next}(e)) :: \kappa \parallel s \parallel H \parallel M \rangle \\ \rightarrow \quad \langle (\sigma \vdash_e e) :: \text{next} :: \kappa \parallel s \parallel H \parallel M \rangle \end{array}$$

4.4 Small-Step Relations for IBop

$$\begin{aligned}
\text{Add} \quad & \langle (+) :: \kappa \parallel n_2 :: n_1 :: s \parallel H \parallel M \rangle \rightarrow \langle \kappa \parallel (n_1 + n_2) :: s \parallel H \parallel M \rangle \\
\text{Mul} \quad & \langle (*) :: \kappa \parallel n_2 :: n_1 :: s \parallel H \parallel M \rangle \rightarrow \langle \kappa \parallel (n_1 \times n_2) :: s \parallel H \parallel M \rangle \\
\text{Div}_0 \quad & \langle (/) :: \kappa \parallel 0 :: n_1 :: s \parallel H \parallel M \rangle \rightarrow \langle \text{raise}(\text{ZeroDivisionError}) :: \square \parallel s \parallel H \parallel M \rangle \\
\text{Div} \quad & \langle (/) :: \kappa \parallel n_2 :: n_1 :: s \parallel H \parallel M \rangle \rightarrow \langle \kappa \parallel (n_1 \div n_2) :: s \parallel H \parallel M \rangle \\
\text{Mod}_0 \quad & \langle (\%) :: \kappa \parallel 0 :: n_1 :: s \parallel H \parallel M \rangle \rightarrow \langle \text{raise}(\text{ZeroDivisionError}) :: \square \parallel s \parallel H \parallel M \rangle \\
\text{Mod} \quad & \langle (\%) :: \kappa \parallel n_2 :: n_1 :: s \parallel H \parallel M \rangle \rightarrow \langle \kappa \parallel (n_1 \% n_2) :: s \parallel H \parallel M \rangle \\
\text{Eq} \quad & \langle (==) :: \kappa \parallel v_2 :: v_1 :: s \parallel H \parallel M \rangle \rightarrow \langle \kappa \parallel \text{equal}(v_1, v_2, M) :: s \parallel H \parallel M \rangle \\
\text{Is} \quad & \langle (\text{is}) :: \kappa \parallel v_2 :: v_1 :: s \parallel H \parallel M \rangle \rightarrow \langle \kappa \parallel \text{is}(v_1, v_2) :: s \parallel H \parallel M \rangle
\end{aligned}$$

$$\begin{aligned}
\text{Lt} \quad & \langle (<) :: \kappa \parallel v_2 :: v_1 :: s \parallel H \parallel M \rangle \\
\rightarrow \quad & \begin{cases} \langle \kappa \parallel b :: s \parallel H \parallel M \rangle & \text{if } \text{lessThan}(v_1, v_2, M) = b \\ \langle \text{raise}(\text{TypeError}) :: \square \parallel s \parallel H \parallel M \rangle & \text{otherwise} \end{cases} \\
\text{Lte} \quad & \langle (<=) :: \kappa \parallel v_2 :: v_1 :: s \parallel H \parallel M \rangle \\
\rightarrow \quad & \begin{cases} \langle \kappa \parallel (b \vee \text{equal}(v_1, v_2, M)) :: s \parallel H \parallel M \rangle & \text{if } \text{lessThan}(v_1, v_2, M) = b \\ \langle \text{raise}(\text{TypeError}) :: \square \parallel s \parallel H \parallel M \rangle & \text{otherwise} \end{cases}
\end{aligned}$$

4.5 Small-Step Relations for Other Instructions

$$\begin{aligned}
\text{IWrite} \quad & \langle \text{write}[a] :: \kappa \parallel v :: s \parallel H \parallel M \rangle \\
\rightarrow \quad & \langle \kappa \parallel s \parallel H \parallel M + (a \mapsto v) \rangle \\
\text{IGetItem} \quad & \langle \text{get-item} :: \kappa \parallel n :: a :: s \parallel H \parallel M \rangle \\
\rightarrow \quad & \begin{cases} \langle \kappa \parallel v_{m+n} :: s \parallel H \parallel M \rangle & \text{if } M(a) = [v_1, \dots, v_m] \wedge -m \leq n < 0 \\ \langle \kappa \parallel v_n :: s \parallel H \parallel M \rangle & \text{if } M(a) = [v_1, \dots, v_m] \wedge 0 \leq n < m \\ \langle \text{raise}(\text{IndexError}) :: \square \parallel s \parallel H \parallel M \rangle & \text{if } M(a) = [v_1, \dots, v_m] \wedge (n < -m \vee m \leq n) \\ \langle \text{raise}(\text{TypeError}) :: \square \parallel s \parallel H \parallel M \rangle & \text{otherwise} \end{cases} \\
\text{ISetItem} \quad & \langle \text{set-item} :: \kappa \parallel n :: a :: v :: s \parallel H \parallel M \rangle \\
\rightarrow \quad & \begin{cases} \langle \kappa \parallel s \parallel H \parallel M + (a \mapsto v'_{m+n}) \rangle & \text{if } M(a) = [v_1, \dots, v_m] \wedge -m \leq n < 0 \\ \langle \kappa \parallel s \parallel H \parallel M + (a \mapsto v'_n) \rangle & \text{if } M(a) = [v_1, \dots, v_m] \wedge 0 \leq n < m \\ \langle \text{raise}(\text{IndexError}) :: \square \parallel s \parallel H \parallel M \rangle & \text{if } M(a) = [v_1, \dots, v_m] \wedge (n < -m \vee m \leq n) \\ \langle \text{raise}(\text{TypeError}) :: \square \parallel s \parallel H \parallel M \rangle & \text{otherwise} \end{cases} \\
& \text{where } v'_k = [v_1, \dots, v_{k-1}, v, v_{k+1}, \dots, v_m]
\end{aligned}$$

IList	$\langle \text{list}[n] :: \kappa \parallel v_n :: \dots :: v_1 :: s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle \kappa \parallel a :: s \parallel H \parallel M + (a \mapsto [v_1, \dots, v_n]) \rangle$ where $a \notin \text{Domain}(M)$
IAppend	$\langle \text{append} :: \kappa \parallel v :: a :: s \parallel H \parallel M \rangle$
$\rightarrow$	$\begin{cases} \langle \kappa \parallel a :: s \parallel H \parallel M + (a \mapsto [v_1, \dots, v_m, v]) \rangle & \text{if } M(a) = [v_1, \dots, v_m] \\ \langle \text{raise}(\text{TypeError}) :: \square \parallel s \parallel H \parallel M \rangle & \text{otherwise} \end{cases}$
IJumpIf	$\langle \text{jmp-if}[\langle \kappa' \parallel s' \parallel H' \rangle] :: \kappa \parallel v :: s \parallel H \parallel M \rangle$
$\rightarrow$	$\begin{cases} \langle \kappa' \parallel s' \parallel H' \parallel M \rangle & \text{if } \text{isTruthy}(v, M) = \text{True} \\ \langle \kappa \parallel s \parallel H \parallel M \rangle & \text{otherwise} \end{cases}$
IJump	$\langle \text{jmp}[c] :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle \kappa' \parallel s' \parallel H' \parallel M \rangle$ where $\langle \kappa' \parallel s' \parallel H' \rangle = H(c)$
IRaise	$\langle \text{raise}(\omega) :: \kappa \parallel s \parallel H \parallel M \rangle$
$\rightarrow$	$\begin{cases} \langle \text{jmp}[\text{raise}] :: \square \parallel s \parallel H \parallel M \rangle & \text{if } \text{raise} \in \text{Domain}(H) \\ (\text{runtime error with message with } \omega\text{'s name}) & \text{otherwise} \end{cases}$
ICall	$\langle \text{call}[n] :: \kappa \parallel v_n :: \dots :: v_1 :: a :: s \parallel H \parallel M \rangle$
$\rightarrow$	$\begin{cases} \langle \kappa' \parallel \text{None} :: \blacksquare \parallel H_{\text{body}} \parallel M_2 \rangle & \text{if } M(a) = \langle \lambda(x_1, \dots, x_n).B, \sigma' \rangle \\ \langle \kappa \parallel a_{\text{iter}} :: s \parallel H \parallel M_4 \rangle & \text{if } M(a) = \langle \lambda*(x_1, \dots, x_n).B, \sigma' \rangle \\ \langle \text{raise}(\text{TypeError}) :: \square \parallel s \parallel H \parallel M \rangle & \text{otherwise} \end{cases}$ where $\begin{aligned} a_1, \dots, a_n &\notin \text{Domain}(M) \\ \{x'_1, \dots, x'_m\} &= \text{locals}(B) \setminus \{x_1, \dots, x_n\} \\ M_1 &= M[a_1 \mapsto v_1, \dots, a_n \mapsto v_n] \\ a'_1, \dots, a'_m &\notin \text{Domain}(M_1) \\ M_2 &= M_1[a'_1 \mapsto \text{None}, \dots, a'_m \mapsto \text{None}] \\ \sigma_{\text{body}} &= \sigma'[x_1 \mapsto a_1, \dots, x_n \mapsto a_n, x'_1 \mapsto a'_1, \dots, x'_m \mapsto a'_m] \\ H_{\text{body}} &= H[\text{return} \mapsto \psi_{\text{return}}] \setminus \{\text{break}, \text{continue}, \text{yield}\} \\ \psi_{\text{return}} &= \langle \kappa \parallel s \parallel H \rangle \\ \kappa' &= (\sigma_{\text{body}} \vdash_B B) :: \text{return} :: \square \\ \psi_{\text{next}} &= \langle \kappa' \parallel \text{None} :: \blacksquare \parallel H_{\text{body}} \rangle \\ a_{\text{cont}} &\notin \text{Domain}(M_2) \\ M_3 &= M_2[a_{\text{cont}} \mapsto \psi_{\text{next}}] \\ a_{\text{iter}} &\notin \text{Domain}(M_3) \\ M_4 &= M_3[a_{\text{iter}} \mapsto \text{iter}[a_{\text{cont}}, 0]] \end{aligned}$
IReturn	$\langle \text{return} :: \kappa \parallel v :: s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle \kappa' \parallel v :: s' \parallel H' \parallel M \rangle$ where $\langle \kappa' \parallel s' \parallel H' \rangle = H(\text{return})$
IYield	$\langle \text{yield} :: \kappa \parallel v :: s \parallel H \parallel M \rangle$
$\rightarrow$	$\langle \kappa' \parallel \langle \kappa \parallel s \parallel H \rangle :: v :: s' \parallel H' \parallel M \rangle$ where $\langle \kappa' \parallel s' \parallel H' \rangle = H(\text{yield})$

$$\begin{aligned}
\text{IIter} \quad & \langle \text{iter} :: \kappa \parallel a :: s \parallel H \parallel M \rangle \\
\rightarrow \quad & \begin{cases} \langle \kappa \parallel a :: s \parallel H \parallel M \rangle & \text{if } M(a) = \text{iter}[_, i] \\ \langle \kappa \parallel a_{\text{iter}} :: s \parallel H \parallel M[a_{\text{iter}} \mapsto \text{iter}[a, 0]] \rangle & \text{if } M(a) = [_] \\ \langle \text{raise}(\text{TypeError}) :: \square \parallel s \parallel H \parallel M \rangle & \text{otherwise} \end{cases} \\
& \text{where } a_{\text{iter}} \notin \text{Domain}(M) \\
\\
\text{INext} \quad & \langle \text{next} :: \kappa \parallel a :: s \parallel H \parallel M \rangle \\
\rightarrow \quad & \begin{cases} \langle \kappa' \parallel s' \parallel H_{\text{next}} \parallel M \rangle & \text{if } v = \text{iter}[a', m] \wedge v' = \langle \kappa' \parallel s' \parallel H' \rangle \\ \langle \kappa \parallel v_m :: s \parallel H \parallel M_{\text{next}} \rangle & \text{if } v = \text{iter}[a', m] \wedge v' = [v_1, \dots, v_n] \wedge m < n \\ \langle \text{raise}(\text{StopIteration}) :: \square \parallel s \parallel H \parallel M \rangle & \text{if } v = \text{iter}[a', m] \wedge v' = [v_1, \dots, v_n] \wedge n \leq m \\ \langle \text{raise}(\text{TypeError}) :: \square \parallel s \parallel H \parallel M \rangle & \text{otherwise} \end{cases} \\
& \text{where } \begin{cases} v = M(a) \\ v' = M(a') \\ \psi_{\text{yield}} = \langle \text{write}[a'] :: \kappa \parallel s \parallel H \rangle \\ \psi_{\text{return}} = \langle \text{drop} :: \text{raise}(\text{StopIteration}) :: \square \parallel s \parallel H \rangle \\ H_{\text{next}} = H'[\text{yield} \mapsto \psi_{\text{yield}}, \text{return} \mapsto \psi_{\text{return}}] \\ M_{\text{next}} = M[a \mapsto \text{iter}[a', m + 1]] \end{cases} \\
\\
\text{IDrop} \quad & \langle \text{drop} :: \kappa \parallel v :: s \parallel H \parallel M \rangle \\
\rightarrow \quad & \langle \kappa \parallel s \parallel H \parallel M \rangle
\end{aligned}$$

Other remaining cases raise a `TypeError` as follow:

$$\langle \kappa \parallel s \parallel H \parallel M \rangle \rightarrow \langle \text{raise}(\text{TypeError}) :: \square \parallel s \parallel H \parallel M \rangle$$

4.6 Auxiliary Definitions

The following auxiliary functions are used in the small-step rules. Note that $A \rightarrow B$ denotes a partial function from set A to set B .

$$\begin{aligned}
& \boxed{\text{equal} : (\mathbb{V} \times \mathbb{V} \times \mathbb{M}) \rightarrow \mathbb{B}} \\
\text{equal}(v, v', M) = & \begin{cases} \text{True} & \text{if } \text{is}(v, v') \\ \text{equal}(M(a), M(a'), M) & \text{if } v = a \wedge v' = a' \\ \text{equal}(v_1, v'_1, M) \wedge \dots \wedge \text{equal}(v_n, v'_n, M) & \text{if } v = [v_1, \dots, v_n] \wedge v' = [v'_1, \dots, v'_n] \\ \text{False} & \text{otherwise} \end{cases} \\
& \boxed{\text{lessThan} : (\mathbb{V} \times \mathbb{V} \times \mathbb{M}) \rightarrow \mathbb{B}} \\
\text{lessThan}(n_0, n_1, M) & = n_0 < n_1 \\
\text{lessThan}(a_0, a_1, M) & = \text{lessThan}(M(a_0), M(a_1), M) \\
\text{lessThan}([v_1, \dots, v_n], [v'_1, \dots, v'_m], M) & = \begin{cases} \text{False} & \text{if } n = m = 0 \\ \text{True} & \text{if } n = 0 < m \\ \text{False} & \text{if } n > 0 = m \\ \text{True} & \text{if } b \\ \text{False} & \text{if } \neg b \wedge \neg b' \\ \text{lessThan}([v_2, \dots, v_n], [v'_2, \dots, v'_m], M) & \text{if } \neg b \wedge b' \end{cases} \\
& \text{where } b = \text{lessThan}(v_1, v'_1, M) \text{ and } b' = \text{equal}(v_1, v'_1, M)
\end{aligned}$$

$$\boxed{\text{is} : (\mathbb{V} \times \mathbb{V}) \rightarrow \mathbb{B}}$$

$$\text{is}(v, v') = \begin{cases} \text{True} & \text{if } v = v' = \text{None} \\ \text{True} & \text{if } v = v' = n \\ \text{True} & \text{if } v = v' = b \\ \text{True} & \text{if } v = v' = a \\ \text{False} & \text{otherwise} \end{cases}$$

$$\boxed{\text{isTruthy} : (\mathbb{V} \times \mathbb{M}) \rightarrow \mathbb{B}}$$

$$\text{isTruthy}(v, M) = \begin{cases} \text{False} & \text{if } v = \text{None} \\ n \neq 0 & \text{if } v = n \\ b & \text{if } v = b \\ \text{isTruthy}(M(a), M) & \text{if } v = a \\ 0 < n & \text{if } v = [v_1, \dots, v_n] \\ \text{True} & \text{otherwise} \end{cases}$$

$$\boxed{\text{locals}(B) : \mathcal{P}(\mathbb{X})}$$

$$\text{locals}(\{S_1; \dots; S_n\}) = \text{locals}(S_1) \cup \dots \cup \text{locals}(S_n)$$

$$\text{locals}(S) = \begin{cases} \{x\} & \text{if } S = x = e \\ \{x_0\} & \text{if } S = \text{def } x_0 (x_1, \dots, x_n) B \\ \text{locals}(B_0) \cup \text{locals}(B_1) & \text{if } S = \text{if } e B_0 \text{ else } B_1 \\ \text{locals}(B_0) \cup \text{locals}(B_1) & \text{if } S = \text{try } B_0 \text{ except } B_1 \\ \text{locals}(B) & \text{if } S = \text{while } e B \\ \emptyset & \text{otherwise} \end{cases}$$

$$\boxed{\text{hasYield}(B) : \mathbb{B}}$$

$$\text{hasYield}(\{S_1; \dots; S_n\}) = \text{hasYield}(S_1) \vee \dots \vee \text{hasYield}(S_n)$$

$$\text{hasYield}(S) = \begin{cases} \text{True} & \text{if } S = \text{yield } e \\ \text{hasYield}(B) & \text{if } S = \text{def } x_0 (x_1, \dots, x_n) B \\ \text{hasYield}(B_0) \vee \text{hasYield}(B_1) & \text{if } S = \text{if } e B_0 \text{ else } B_1 \\ \text{hasYield}(B_0) \vee \text{hasYield}(B_1) & \text{if } S = \text{try } B_0 \text{ except } B_1 \\ \text{hasYield}(B) & \text{if } S = \text{while } e B \\ \text{False} & \text{otherwise} \end{cases}$$